

However, utilizing a specialized model for hallucination correction offers a stronger alternative - this type of model is trained specifically to take an output from RAG that is suspected to be a hallucination and “correct” it based on the retrieved context.

As shown in [Figure 2-3](#), a hallucination correction model is invoked when a hallucination correction model has identified the RAG response as hallucinated.

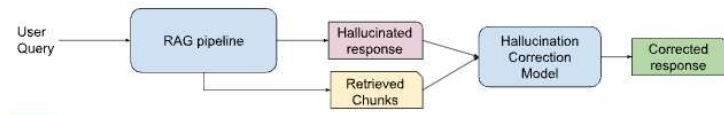


Figure 2-3. hallucination correction flow

With that hallucinated response and the source chunks that were retrieved during the RAG flow as input, the model’s output is a *corrected* response that can be sent to the user instead of the hallucinated one.

Combining the strength of a hallucination detection model with a hallucination correction model in your RAG pipeline allows you to mitigate RAG hallucinations even further than basic RAG allows, making your RAG trusted and reliable.

Before we conclude this Chapter we need to discuss one more, often neglected, aspect of building advanced RAG applications - creating a great user experience. Let’s dive in.

Building A Great RAG User Experience

Creating a great user experience in RAG requires careful consideration of how users interact with an AI application that both retrieves and generates information, and how to carefully present this information to the user so that it is most useful to them in performing the task in front of them.

Let’s look at some key aspects for creating amazing user experiences in RAG applications.

Considerations in RAG User Experience

To create a great UI for a RAG application, you should consider the following three aspects: how to capture user input, how to present the results from the RAG pipeline, and how to obtain user feedback.

Capture of User Input

Users approach a RAG application with a goal in mind, whether that’s finding the answer to a question or completing a specific task. The user’s first step towards this goal is to express their goal clearly to the RAG application. To capture this information, the RAG application must optimize for user expression.

There are a few important considerations when designing this input interface:

Users think in natural language. To express themselves comfortably, users must be able to interact with the RAG application using their language of choice in a natural, unstructured manner. The RAG application can enable this with a prominent input for entering text, by supporting file uploads such as images and PDFs, by accepting voice input, or a combination of these. The point is to offer an input interface that encourages conversational interaction.

Query Refinement

The user interface can provide tools or suggestions that help users refine their queries. This could include auto-suggest features or example queries, which helps users get more precise and relevant results.

Multi-turn and Chat History

Users of RAG applications expect to be able to converse with an AI assistant continuously in multiple “turns”, where the AI assistant has memory of the conversation and uses the full context of the conversation to better address the user’s requests. The UI should reflect this, allowing users to see the history of their interaction and easily refer back to previous queries or responses.

Figure 2-4 shows an example of a RAG question-answering application grounded in news articles from CNN, CNBC, NPR, Fox News, and the BBC. As you can see, the user interface has a prominent box for users to type in their queries in natural language, and a user can even select whether they want the query to run against all news sources or just a single one. Additionally, there are four “suggested queries” the user can click on - those can be useful queries on their own, but importantly also demonstrate the type and format of queries the user may try.

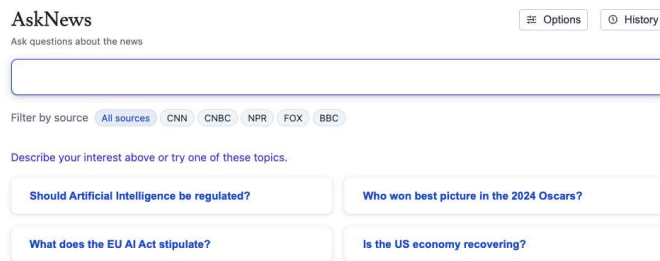


Figure 2-4. Example user interface for a RAG question-answering application

You should customize each of the considerations above to specific use-cases. For example, consider a chatbot designed to support customer service agents at an airline. A good base design for this type of chatbot should certainly include a prominent input area, suggested queries, and multi-turn chat, but you can go a step further. What if the history of all conversations with a specific customer can be used to generate suggested queries that anticipate their needs based on what they’ve requested in the past? This type of personalization helps the end user of your RAG application accomplish their goals even more quickly.

Presentation of Results

After the question or task is clear, the RAG application continues to process the input and come up with its response. The output of RAG includes three main components: the generated response, the source documents (or chunks), and additional metadata like a hallucination or confidence score.

Here are some important points you should consider regarding presenting that response in a coherent and effective way:

Integrated Response

The AI-generated response and the retrieved information should be presented in a digestible manner. Avoid simply dumping a list of sources alongside a text response. Instead, integrate the response, citation sources and other metadata seamlessly into the flow of the generated text. Use clear visual cues to distinguish between the AI-generated content, the retrieved information and the metadata. This could involve different font styles, colors, or background treatments. A well-designed visual hierarchy helps users parse the information quickly and understand its origin.

Source Attribution

You need to show users where the information that was used to generate the response came from – its lineage. RAG applications pull data from various sources, so providing clear citations or links to these sources builds trust and enables users to verify the information. This transparency also helps users understand the context and potential biases of the retrieved data, and they can click on these references to go to the source and obtain additional information or context. You might consider highlighting the specific passages that are most relevant to the user's query. This makes it easier for users to quickly find the information they're looking for and understand why it was included.

Process Explanation

Briefly “explaining” the RAG process can be beneficial and enhance the user experience. Users should have a general understanding of how the application works – that it's retrieving information to enhance the AI's response. You can do this through subtle UI elements, like loading indicators that show data being fetched, or a short, optional explanation of what's happening behind the scenes.

[Figure 2-5](#) demonstrates some of these principles.



Figure 2-5. Presentation of results of a question-answering RAG application

We can see that the response to the user’s question is presented along with citations integrated into the response, and linked below it (in this case we only show two citations in the screenshot out of the four used by the RAG application). This allows users to better understand the source documents that the response was grounded on. Furthermore, the “progress report” tab is used for process explanation, helping the user understand how the response is generated, as it’s being generated.

User Control and Feedback

As the user interacts with the RAG applications, it is sometimes beneficial to provide ways for the user to control how the application works and provide feedback. Some options include:

Control over Sources

In some cases, it might be beneficial to give users some control over the sources that the RAG application uses. This could involve allowing them to prioritize certain sources, exclude others, or even add their own data sources. For example in a knowledge management RAG application, the answer may be based on documents from Google Drive, Slack, Notion or JIRA, and a user may want to get a response that is based only on one of those sources (for example only Google Drive) instead of all of them.

Feedback Mechanisms

Provide clear and easy ways for users to provide feedback on the quality of the responses and the relevance of the retrieved information, such as thumbs up/down buttons, or the ability to highlight specific parts of the response and provide comments. If you choose to provide such mechanisms (and we highly recommend that you do) - make sure you not only capture this feedback in the user interface, but also store it someplace in your RAG backend as it will be useful for RAG evaluation.

The user interface should gracefully handle situations where the RAG system fails to retrieve relevant information or generates an inaccurate response. In those cases, provide informative error messages and suggest alternative ways for the user to find the information they need.

With precise control over data source, the user feels more empowered to navigate the results and guide the RAG application better to their intent.



Multi-Modal User Interfaces

If your application supports multi-modal inputs like images or videos, you need to consider how to present those elements to the user.

For example, if a diagram or image is returned as part of the retrieval process used in the final generation, you need to design your user interface so that it presents that diagram or image to the user as a valid citation in a way that is easy to consume and understand.

Take a look at an example in this [blog post by Microsoft](#). The multi-modal RAG application does not only provide a link to the image citation but actually displays the image as part of the response presentation.

Tools and Reference Implementations

Let's look at some open source tools and reference implementations that demonstrate the concepts we discussed above: capturing user input, presentation of results and providing users with control and feedback mechanisms.

Assistant UI

[Assistant UI](#) is an open source Typescript/React library for AI Chat. As you can see in [this demo](#) of Assistant UI with LangChain, it implements many of the suggestions from “Considerations in RAG User Experience” including:

- User input is captured in a well with a search box as well as example queries.
- The output is also well presented, including a progress bar, citations and clear presentation of the response.
- The user can click the thumbs up or thumbs down icons at the end of the response to provide feedback.

Streamlit and Gradio

[Streamlit](#) is a popular open-source Python framework favored by data scientists and AI/ML engineers for its simplicity in creating interactive web applications directly from Python scripts. While initially designed for data visualization and dashboarding, it has become a common choice for building chatbot user interfaces (using dedicated chat elements like `st.chat_input` for capturing user queries and `st.chat_message` for displaying conversation history in a familiar messaging format).

Because Streamlit's primary goal is broader data application development, its default styling and layout capabilities might result in user interfaces that seem less polished compared to more dedicated chat interfaces like Assistant UI. However, its ease of use, rapid development cycle, and pure Python environment make it a great tool for prototyping, internal tools, or applications where speed of development outweighs the need for extensive UI customization.

Streamlit's ecosystem allows for extensibility through custom components, enabling developers to add features like user feedback mechanisms (e.g., thumbs up/down buttons) to enhance the chat experience.

Similar to Streamlit in its Python-centric approach, [Gradio](#) is another open-source Python library specifically focused on creating user interfaces for machine learning models, APIs, and data science workflows. Developed by Hugging Face, Gradio excels at quickly generating demos and shareable web applications. For chatbot development, Gradio's `gr.ChatInterface` provides a complete, pre-built chat UI with minimal code, often requiring just a function that processes user input and returns the chatbot's response.

Vectara Answer

[Vectara-answer](#) is an open-source RAG user interface specifically designed for question-answering (i.e. single question / single answer) applications that connect natively with the Vectara platform.

Built using React and TypeScript, vectara-answer provides a clean and functional user experience out-of-the-box, and serves as a ready-to-deploy reference implementation, showcasing how developers can build a front-end experience that implements many of the important considerations we discussed earlier in this section:

- The UI presents a prominent and easy to use input box to the user to capture their query, as well as curated questions that can provide some examples.
- When a query is issued, the “progress report” component reports back to the user on each in the RAG process: retrieval of results and generation of summary.
- The generated summary includes citations. Those are also clickable, so that it's easy for the user to check the information sources that were used to generate the response.
- The response includes a “hallucination badge” representing the hallucination score of the response, providing the user with more context about the likelihood of the response to have hallucinations.

[Figure 2-4](#) and 3-5 from earlier in the chapter show an example application built using vectara-answer.

Conclusion

Continuing our discussion about the basics of building RAG from Chapter 2, in this chapter we focused on more advanced RAG techniques that you need to implement when your RAG scales.